CS 201: Data Structures - Binary Search Trees (Lecture 14)
Instructor: Dr. Elena Rostova

1. Motivation: The Dynamic Ordered Set Problem
In computer systems and database design, we frequently require a data structure
that maintains a dynamic collection of keys under Search, Insert, and Delete.
Arrays offer either fast search (sorted) or fast insert (unsorted), but not both.
Linked lists allow O(1) pointer updates at the head, but lack random access for binary search.
A Binary Search Tree achieves O(log N) search and updates simultaneously when balanced.

2. The Binary Search Tree Property
Let x be any node in binary tree T.
- For any node y in left subtree of x: key(y) < key(x)
- For any node y in right subtree of x: key(y) >= key(x)

Crucial Invariant Scope: This condition applies globally across all descendant subtrees, not merely between immediate parent and children.
In-order traversal visits nodes in strictly non-decreasing sorted order.
Tree sort runs in Theta(N) time by performing an in-order walk over a constructed BST.

3. Operations and Complexity
- Tree-Search(x, k): Discards half the tree at each level. Cost: O(h).
- Tree-Insert(T, z): Traces path from root to leaf boundary with trailing pointer. Cost: O(h).
- Tree-Successor(x): Smallest key greater than x.key. Concludes at 18:42 timestamp in lecture.
- Tree-Delete(T, z): 3 structural cases (leaf, single child, two children with successor substitution).
- Tree Degeneracy: Inserting sorted keys [1..N] produces height h = N-1, collapsing operations to O(N). Balanced AVL and Red-Black trees prevent degeneracy via local O(1) tree rotations.